

Software Requirements Specification

for

**Hotel Reservation and Guest Services
Management System**

Version 1.0 approved

Prepared by Group 33

July 5, 2026

Table of Content

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Document Conventions.....	4
1.3 Intended Audience and Reading Suggestions.....	4
1.4 Product Scope.....	5
1.5 References.....	5
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions.....	6
2.3 User Classes and Characteristics.....	7
2.4 Operating Environment.....	9
2.5 Design and Implementation Constraints.....	11
2.6 User Documentation.....	13
2.7 Assumptions and Dependencies.....	14
3. External Interface Requirements.....	15
3.1 User Interfaces.....	15
3.2 Hardware Interfaces.....	22
3.3 Software Interfaces.....	24
3.4 Communications Interfaces.....	24
4. System Features.....	26
4.1 Room & Branch Management	27
4.2 Booking Management	27
4.3 Guest Management	28
4.4 Check-In and Check-Out	29
4.5 Chargeable Service Tracking	29
4.6 Billing & Payments	30
4.7 Management Reports	31
5. Other Nonfunctional Requirements.....	31
6. Other Requirements	33
Appendix A: Glossary	33
Appendix B: Analysis Models	34
Appendix C: To Be Determined List	36
Appendix D: Team Members	37

Revision History

Name	Date	Reason For Changes	Version

1.0 Introduction

1.1 Purpose

The purpose of this document is to present a detailed description of the Hotel Reservation and Guest Services Management System (HRGSMS) being developed for SkyNest Hotels. It explains the purpose and features of the system, the interfaces of the system, what the system will do, the constraints under which it must operate, and how the system will respond to the actions of its users. The primary emphasis of this project is the correctness, consistency, and reporting capability of the underlying database; a user interface is required but is secondary to the data model. This document is intended for both the stakeholders (SkyNest Hotels management) and the developers of the system.

1.2 Document Conventions

This document adheres to the IEEE Software Requirements Specification (SRS) standards. It follows the IEEE guidelines for font and text styles, use of tables and figures, change history, formatting and page layout, and review and approval processes.

The following conventions apply throughout:

- Bold text indicates defined terms or section titles.
- REQ-X.Y identifiers uniquely tag each functional requirement (X = feature number, Y = requirement number within that feature).
- TBD denotes items not yet finalised (see Appendix C for the consolidated list).
- "Shall" denotes a mandatory requirement; "should" denotes a recommended one.
- All monetary values are in Sri Lankan Rupees (LKR) unless stated otherwise.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following stakeholders involved in the development and deployment of the SkyNest Hotel Reservation and Guest Services Management System:

- **Project Supervisor:** Focus on the system's purpose, scope, high-level requirements, and IEEE standard compliance.
- **Database Designers:** Review data structures, ACID compliance, normalization (BCNF), and schema design.
- **Developers (Backend/Frontend):** Review system features, API specifications, and integration logic.
- **QA Engineers:** Use functional and non-functional requirements to create test cases.

- **Hotel Management:** Review management reports, occupancy metrics, and operational dashboards.
- **Receptionists:** Focus on check-in/check-out workflows, booking management, and billing processes.
- **Guests:** Focus on room browsing, booking flows, service requests, and bill tracking.

1.4 Product Scope

SkyNest Hotels is a regional hotel chain operating in Sri Lanka, with branches in Colombo, Kandy, and Galle. Each branch currently relies on an outdated desktop-based system for bookings and service tracking, resulting in frequent overbookings, billing delays, and manual errors. HRGSMS replaces this legacy setup with a single, centrally hosted database system shared across all three branches. The primary deliverable is a correct, consistent, ACID-compliant data model; a functional web interface is required to demonstrate the system but is secondary to the database design itself. Full functional detail is provided in Section 4.

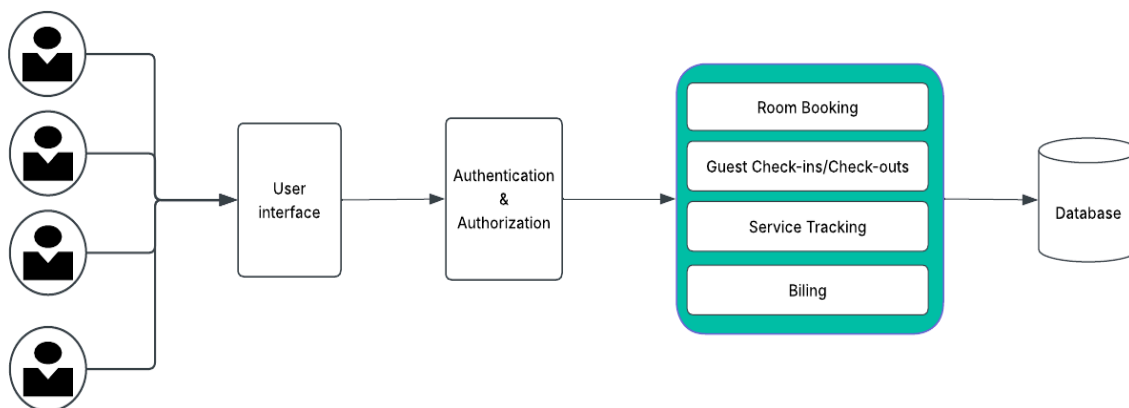
1.5 References

1. IEEE. IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
<http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf>
2. MySQL 8.0 Reference Manual, Oracle Corporation,
<https://dev.mysql.com/doc/refman/8.0/en/>.
3. How to write a Software Requirement Specification (SRS) Document
<https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document>

2.0 Overall Description

2.1 Product Perspective

HRGSMS is a new, self contained database system for SkyNest Hotels, intended to replace the outdated, desktop based booking and service tracking system currently used independently at each branch. It integrates user facing interfaces with secure authentication, allows management to run chain wide reports , and centralized data management. The system supports multiple user roles, including guest, front desk staff, hotel branch manager and system administrator(DBA), each interacting with specific features through a unified interface.



2.2 Product Functions

2.2.1 Room and Branch Management

- Maintaining branch records (location, contact, manager) and room type definitions (capacity, base daily rate, amenities).
- Registering individual rooms under a branch and room type, with status tracked as Available, Occupied, or Under Maintenance.
- Viewing current room availability per branch, filterable by room type and status.

2.2.2 Guest and Booking Management

- Registering both individual and corporate guests, with uniqueness enforced on NIC/passport number.
- Creating bookings with guest, room, dates, and payment method, while preventing any overlapping booking for the same room.
- Managing booking status transitions (Booked → Checked-In → Checked-Out, or Cancelled), including a no-show policy for cancellations after check-in.
- Tracking completed bookings per guest to determine loyalty discount eligibility.

2.2.3 Check-In and Check-Out

- Checking guests in, automatically updating room status to Occupied via trigger.
- Checking guests out through a billing procedure that blocks the transition if an outstanding balance remains, and automatically updates room status to Available.
- Applying a late checkout surcharge (50% of the daily rate) when applicable.

2.2.4 Chargeable Services

- Maintaining a service catalogue (Room Service, Spa, Laundry, Minibar, Airport Transfer, Extra Bed) with current unit pricing.
- Logging service usage against active (Checked-In) bookings only, preserving the price at the time of use regardless of later catalogue price changes.
- Providing a service usage breakdown by category per booking.

2.2.5 Billing and Payments

- Calculating itemized bills combining room charges, service charges, applicable loyalty discount, and tax (15% VAT + 1% Tourism Levy).
- Recording full or partial payments, automatically recalculating the outstanding balance and flagging unpaid bookings.
- Blocking checkout completion until the outstanding balance reaches zero.

2.2.6 Management Reporting

- Room occupancy reports by date/branch, guest billing summaries with outstanding balance flags, service usage breakdowns, monthly branch revenue reports, and top-services-by-revenue reports.

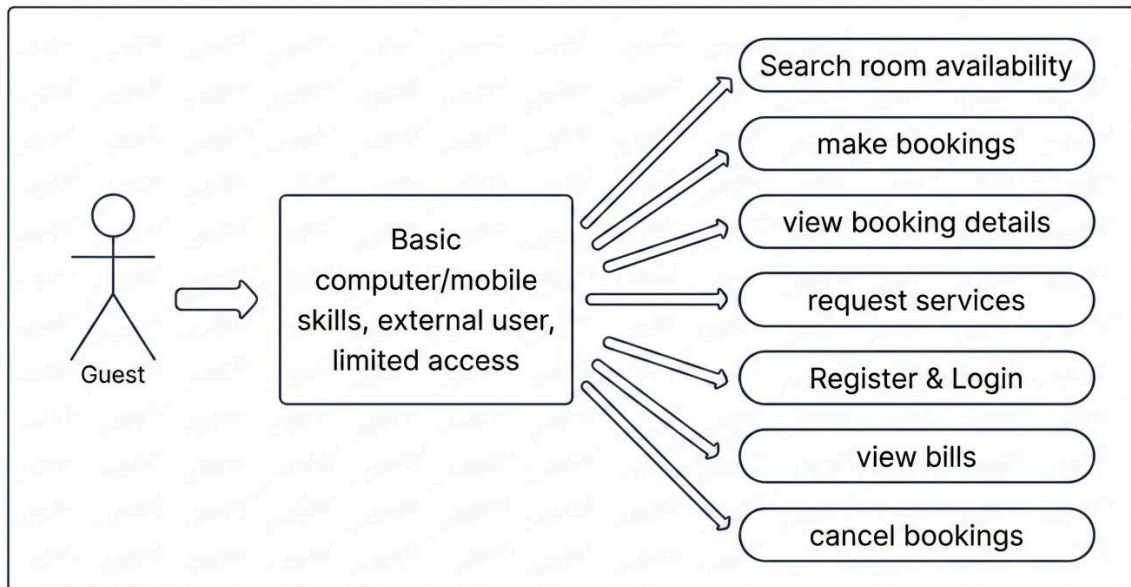
2.2.7 Data Integrity and Transaction Safety

- Enforcing ACID compliance across all multi-step operations (booking, check-in/out, billing, payment) through transactional stored procedures, constraint-based validation, row-locking on concurrent booking attempts, and DBMS-level durability guarantees.

2.3 User Classes and Characteristics

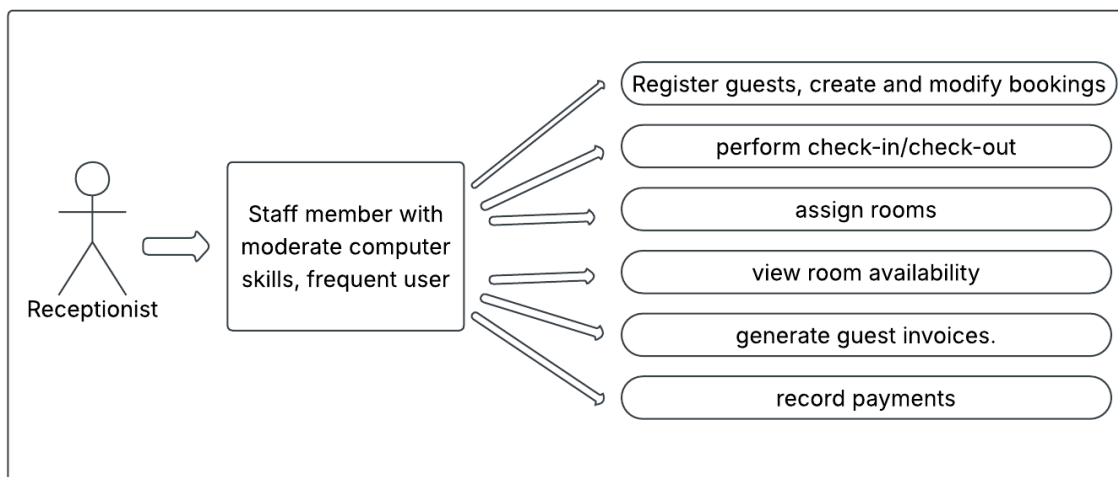
Guest

Authenticated visitors or registered customers, typically first-time or infrequent users accessing the system from personal devices. Low assumed technical proficiency; requires a simple, self-explanatory browsing and booking flow with minimal steps.



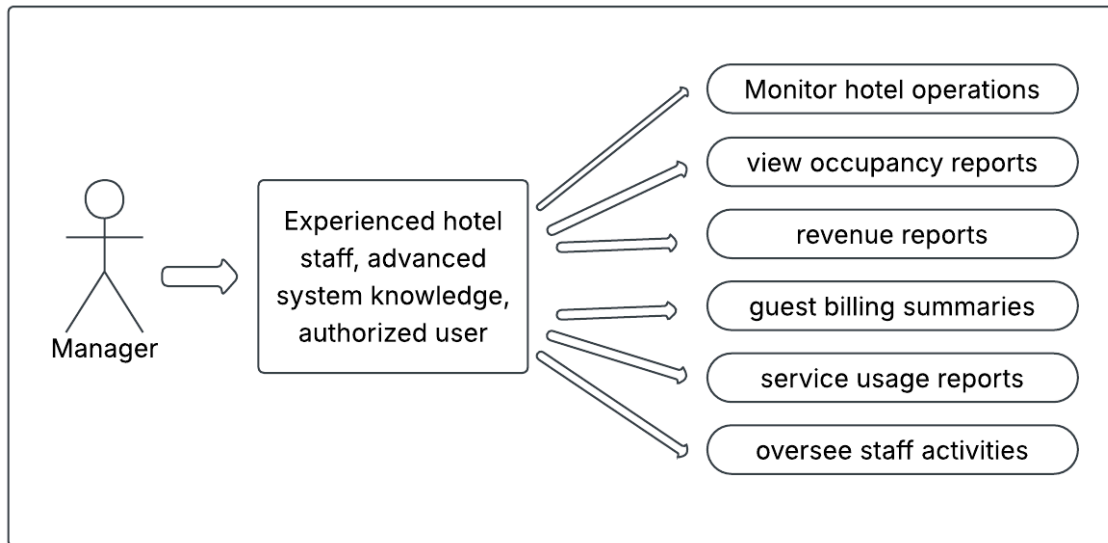
Receptionist

Front-desk staff, frequent daily users with moderate computer skills. Operates under time pressure during check-in/out, so requires fast, error-tolerant forms with clear status feedback. Branch-scoped access only.



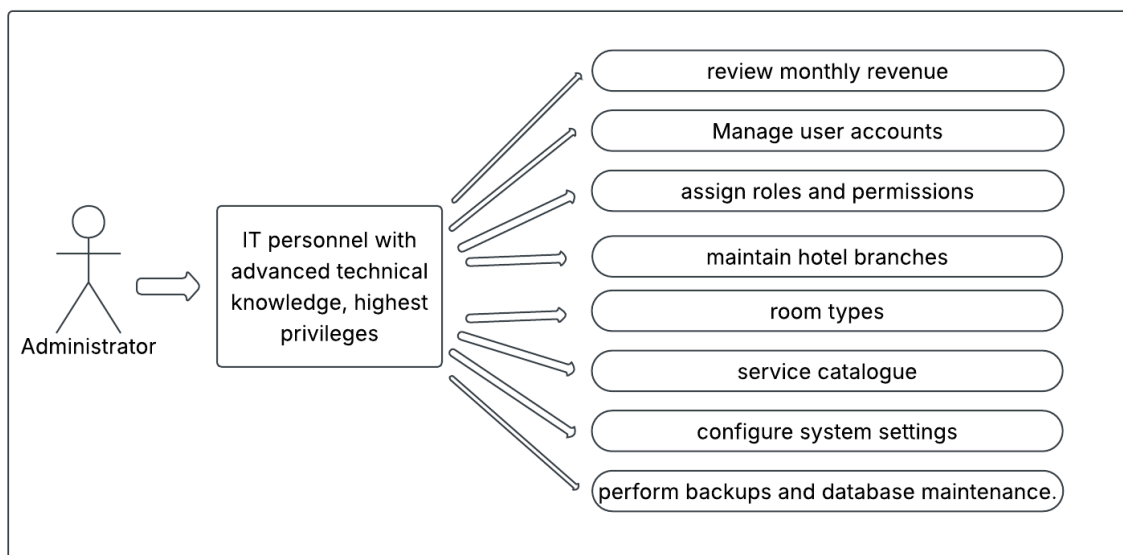
Manager

Experienced hotel staff with advanced system familiarity, but a read-heavy usage pattern — mostly reviewing reports rather than entering data. Branch-scoped access.



Administrator

IT personnel with the highest technical proficiency and full system access across all branches, including account provisioning and system configuration.



2.4 Operating Environment

The HRGSMS is a cloud-hosted, web-only application. All components run on managed free-tier cloud services; no on-premise installation is required at any branch. The table below specifies each component of the deployment environment.

Component	Specification / Choice	Rationale
DBMS	PostgreSQL 15+ Neon.tech — serverless managed PostgreSQL	ACID-compliant; supports stored procedures, triggers, window functions, CTEs, row-level security (RLS), and MVCC. Neon.tech provides a free-tier cloud instance with branching support for development/production isolation.
Backend Technologies	Python 3.11+ FastAPI psycopg2 (raw SQL — no ORM)	FastAPI provides async request handling and automatic OpenAPI documentation. psycopg2 gives direct control over SQL execution, which is required for stored procedure calls and transaction management.
Frontend Technologies	HTML5 CSS3 Vanilla JavaScript (served via Vercel)	Lightweight, no build toolchain required. Vercel provides free static hosting with automatic GitHub deployment.
Backend Hosting	Render.com (free tier — Python web service)	Supports Python WSGI/ASGI applications; automatic deployment from GitHub; always-on within free-tier constraints.
Version Control / CI	GitHub (monorepo) · GitHub Actions (CI/CD pipeline)	Single repository for DB schema, backend, and frontend. GitHub Actions runs schema migration checks and backend tests on every push to main.
Client Devices	Desktop / laptop browsers — Chrome, Firefox, Edge (latest)	Web-only; no mobile-native or desktop app required in initial release.
Network / Security	HTTPS (TLS 1.2+) enforced at Vercel and Render; Neon.tech connection over SSL	All traffic encrypted in transit. Database credentials stored as environment variables; never in source code.
Time Zone	Asia/Colombo (UTC+5:30)	All timestamps stored in UTC in PostgreSQL; converted to Asia/Colombo for UI display.
Currency	Sri Lankan Rupee (LKR)	All monetary columns use NUMERIC(12,2); application enforces two-decimal-place display.

2.5 Design & Implementation Constraints

The following constraints are binding on all design and implementation decisions. They are not negotiable without supervisor approval.

Database-First Architecture

- All business logic — double-booking prevention, billing calculation, room status transitions, partial payment flagging, loyalty discount eligibility — shall reside in the database layer as stored procedures, functions, or triggers.
- The FastAPI backend shall act as a thin routing layer only: it receives HTTP requests, calls the appropriate stored procedure or parameterised SQL statement via psycopg2, and returns the result as JSON. No business logic shall be re-implemented in Python.
- The PoC UI shall not perform any calculation or validation independently of the database.

Double-Booking Prevention

- Overlapping reservation detection shall be enforced by a BEFORE INSERT / BEFORE UPDATE trigger on the booking table at the database level.
- Concurrent booking attempts on the same room shall be serialised using SELECT ... FOR UPDATE within the booking stored procedure to prevent race conditions. The transaction isolation level for booking creation shall be SERIALIZABLE.

ACID Compliance & Data Integrity

- **Atomicity** - All multi-step operations (check-in, checkout, billing, payment) wrapped in BEGIN...COMMIT / ROLLBACK transaction blocks inside stored procedures. On any error, the entire operation rolls back.
- **Consistency** - CHECK constraints (status enums, date ordering, non-negative amounts), UNIQUE constraints (NIC, room-date overlaps via trigger), FOREIGN KEY constraints with appropriate CASCADE rules, and BEFORE triggers enforcing business rules before DML.
- **Isolation** - Concurrent booking attempts on the same room are serialised using SELECT ... FOR UPDATE locking on the availability check inside the booking procedure. Isolation level: READ COMMITTED minimum; SERIALIZABLE for booking creation.

- **Durability** - Enforced by the DBMS write-ahead log (MySQL InnoDB redo log / PostgreSQL WAL). All committed transactions survive server crashes. Daily logical backups provide point-in-time recovery.

No ORM Policy

- psycopg2 is used for all database access. SQLAlchemy, Django ORM, and similar abstraction layers are explicitly excluded. This constraint exists to ensure all SQL — including stored procedure calls — is written and reviewed by the team directly.

Price Snapshot Integrity

- The unit_price column on service_usage shall store the price at the time of logging (snapshot), not a foreign-key reference to the live catalogue price. This ensures historical billing accuracy when catalogue prices are updated.
- Similarly, rate_at_booking on the booking table stores the room daily rate at the time of booking creation.

Security Constraints

- The application database user (app_user) shall have no DDL privileges. Schema creation and migration are performed by a separate DBA-level account.
- All stored procedures execute with SECURITY DEFINER rights. The application cannot access base tables via ad-hoc DML queries; all access is through procedures.
- API endpoints require JWT authentication. Role-based access control (RBAC) maps the five system roles — Public Visitor, Guest, Staff, Manager, Admin — to permitted API operations.
- Guest NIC/Passport numbers are treated as PII and shall not appear in application log files or error responses.

Tax and Financial Constraints

- Room and service charges are subject to a combined 16% tax (15% VAT + 1% Tourism Levy per the Inland Revenue Act No. 24 of 2017) applied to the net amount after discounts, and a 5% loyalty discount for guests with 5+ completed bookings. The exact computation is specified in REQ-6.2–REQ-6.3.

Indexing Constraint

- Every index created must be justified by an identifiable query workload; every high-cardinality column deliberately left unindexed must also be justified in the written deliverable. The concrete index list and the queries each one accelerates are documented in section 5.1.

2.6 User Documentation

The following documentation artefacts shall be produced and submitted alongside the system:

Document	Audience	Contents
Database Schema Documentation	Developers, Examiner	Entity-Relationship Diagram (Lucidchart); normalised schema definitions for all 9 tables; data dictionary specifying column names, data types, constraints, and default values; normalisation summary (BCNF justification per table).
Stored Procedure & Trigger Reference	Developers, Examiner	Name, purpose, input parameters, output/return values, and exception-handling behaviour for each stored procedure, function, and trigger. Includes the ACID property each object enforces.
API Contract Document	Frontend Developers, QA	OpenAPI 3.0 specification (auto-generated by FastAPI) covering all endpoints: HTTP method, URL path, request body schema, response schema, authentication requirement, and the stored procedure invoked. To be finalised before backend development begins.
Sample Data Script	Developers, Examiner	PostgreSQL-compatible SQL INSERT statements populating all required dummy data: 3 branches, 10+ rooms, 6 services, 5 guests, 8 bookings (covering all statuses), 10+ service usage records, 3+ partial payment scenarios, and edge-case rows for testing business rules.
PoC UI User Guide	Front Desk Staff, Examiner	Screen-by-screen walkthrough of the seven PoC UI screens. Documents each form field,

Document	Audience	Contents
		validation message, and the database procedure invoked on submission.
Report Specifications	Manager, Finance Staff, Examiner	SQL queries (or stored procedure calls) for each of the five management reports, with annotated expected output and column descriptions.
Deployment Guide	DBA, Developers	Step-by-step instructions for provisioning the Neon.tech PostgreSQL database, deploying the FastAPI backend to Render.com, and deploying the frontend to Vercel. Includes environment variable configuration and GitHub Actions CI/CD setup.

2.7 Assumptions & Dependencies

2.7.1 Assumptions

- All three hotel branches (Colombo, Kandy, Galle) operate in Sri Lankan Rupees (LKR). No multi-currency support is required in the initial release.
- Room rates are fixed per room type and do not vary by season, day-of-week, or occupancy level in the initial release. Rate seasonality is deferred to a future phase (see TBD-5).
- Each booking covers exactly one room. Multi-room bookings under a single booking reference are out of scope (TBD-6).
- A single guest profile may hold multiple concurrent bookings across different branches.
- Service prices in the catalogue are recorded at the time of service usage (unit_price_at_usage snapshot). Catalogue price changes do not retroactively affect posted service charges.
- Walk-in guests who do not create a portal account are registered directly in the guest table by front desk staff. A portal account (user_account) is optional for guests.
- When a portal account is created by a guest, the system matches it to an existing guest profile via email address. If no match is found, a new guest profile is created simultaneously.
- Staff, Manager, and Admin roles never have an associated guest profile (guest_id IS NULL in user_account for these roles).
- Late checkout time tracking relies on the actual_checkout_time field recorded by staff at the point of checkout. No automated IoT or POS integration is assumed.

- Tax rates (15% VAT + 1% Tourism Levy) are fixed for the duration of the initial release. Any change requires a schema-level update to the billing stored procedure.
- No external payment gateway integration is required. Payments are recorded manually via the billing screen; the system tracks payment method (Cash / Card / Bank Transfer) but does not process card transactions.
- Email notifications (booking confirmations, payment receipts) are out of scope for the initial release.

2.7.2. Dependencies

- Neon.tech (PostgreSQL hosting): The entire database layer depends on Neon.tech service availability. Neon.tech free tier provides sufficient storage and compute for the project scope. Any service outage directly affects system availability.
- Render.com (backend hosting): The FastAPI backend deployment depends on Render.com free-tier Python web service. The free tier may experience cold-start latency (~30 seconds) after periods of inactivity; this is acceptable for a PoC but must be noted in the demonstration.
- Vercel (frontend hosting): The PoC UI depends on Vercel's free-tier static hosting. No server-side rendering is required.
- GitHub Actions (CI/CD): Automated deployment pipelines depend on GitHub Actions free-tier minutes. The pipeline runs schema migration checks and backend unit tests on every push to the main branch.
- psycopg2 / FastAPI ecosystem: The backend depends on the psycopg2 library for PostgreSQL connectivity and FastAPI for HTTP routing. These are stable, well-maintained libraries with no anticipated compatibility issues against Python 3.11+ and PostgreSQL 15+.
- PostgreSQL 15+ feature set: The reporting queries and RLS policies depend on PostgreSQL-specific features including window functions, CTEs, row-level security, and MVCC. MySQL is not a compatible substitute without significant procedure rewrites.
- Dummy data availability: The PoC UI demonstration depends on the sample data script being executed against the production Neon.tech database before the demonstration session.

3.0 External Interface Requirements

3.1 User Interfaces

The Hotel Reservation and Guest Services Management System (HRGSMS) provides a web-based interface for guests to browse and book rooms, and for receptionists, managers, and admins to manage bookings, services, billing, and reporting through role-specific dashboards. The UI follows a consistent hospitality-branded design across all screens, with access and available actions determined by the logged-in user's role.

The following screens illustrate the high-level interface design and how each user role interacts with the system:

Logical Characteristics

1. User Interface Design

- Follows a clean, hospitality-branded visual style using a navy and gold color scheme, reflecting the SkyNest Hotels brand identity.
- Consistent layout across guest-facing and staff-facing screens: a top navigation bar for guests, and a dashboard-style layout with summary cards and action tables for staff.
- Rounded cards and soft shadows are used throughout to give the interface a modern, approachable feel appropriate for a hospitality product.

2. Screen Layout Constraints

- Responsive design, adapting to desktop, tablet, and mobile screen sizes.
- Minimum supported resolution: 320px (mobile) to 1920px (desktop).
- Dashboards (receptionist, manager, admin) are optimized primarily for desktop/tablet use, since staff are expected to operate the system from front-desk or office computers.
- Accessibility compliance for color contrast and readable font sizes.

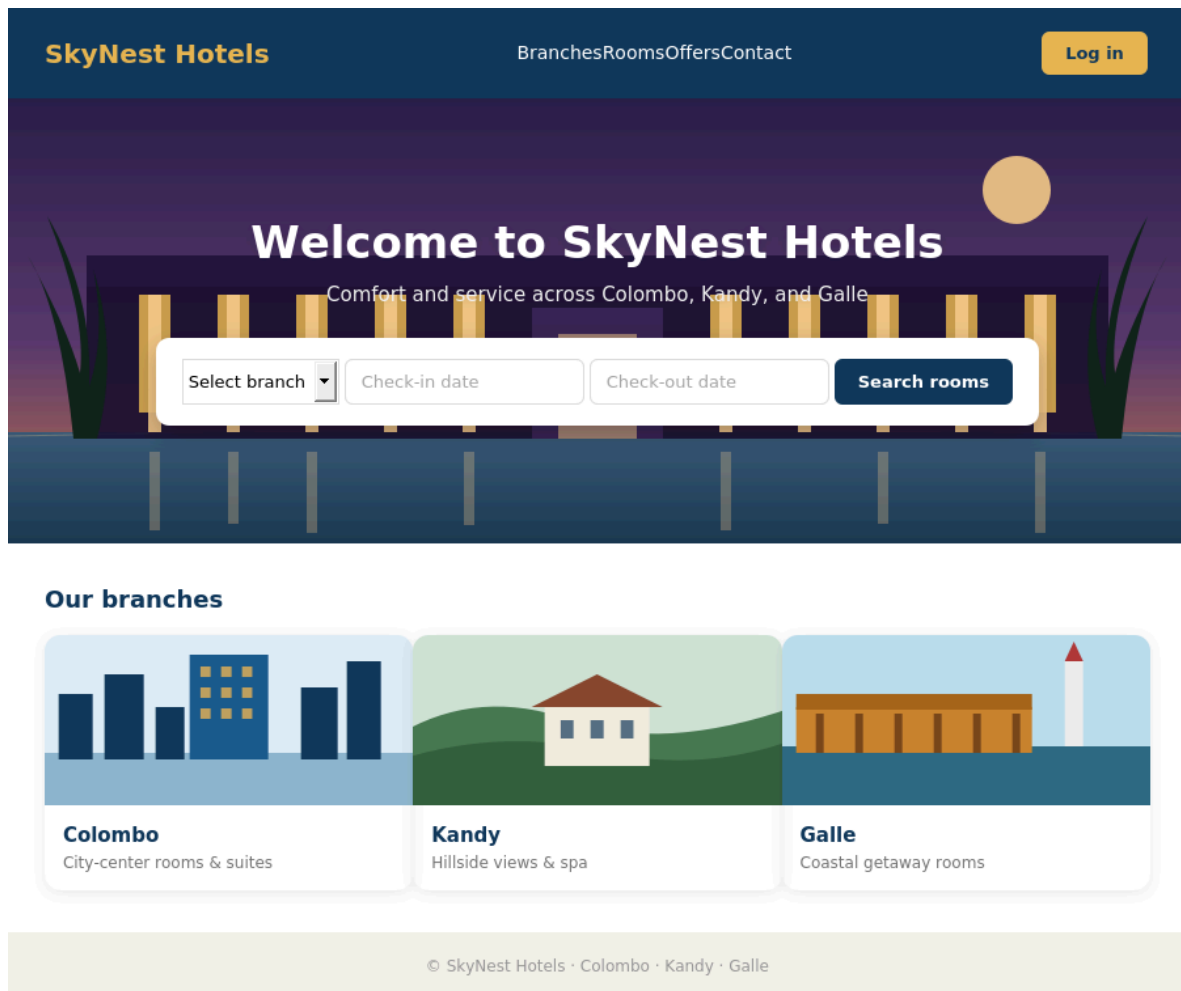
3. Standard Buttons & Functions

- Global elements: every guest-facing screen includes a top-right search bar and login control; every staff dashboard includes a role label (e.g. "— Colombo branch") and a notification icon.
- Role-based access: the system identifies the logged-in user's role (guest, receptionist, manager, admin) and routes them to the appropriate dashboard automatically.
- Error messages are clear and actionable (e.g. invalid login credentials, failed booking due to date overlap).

Software Components

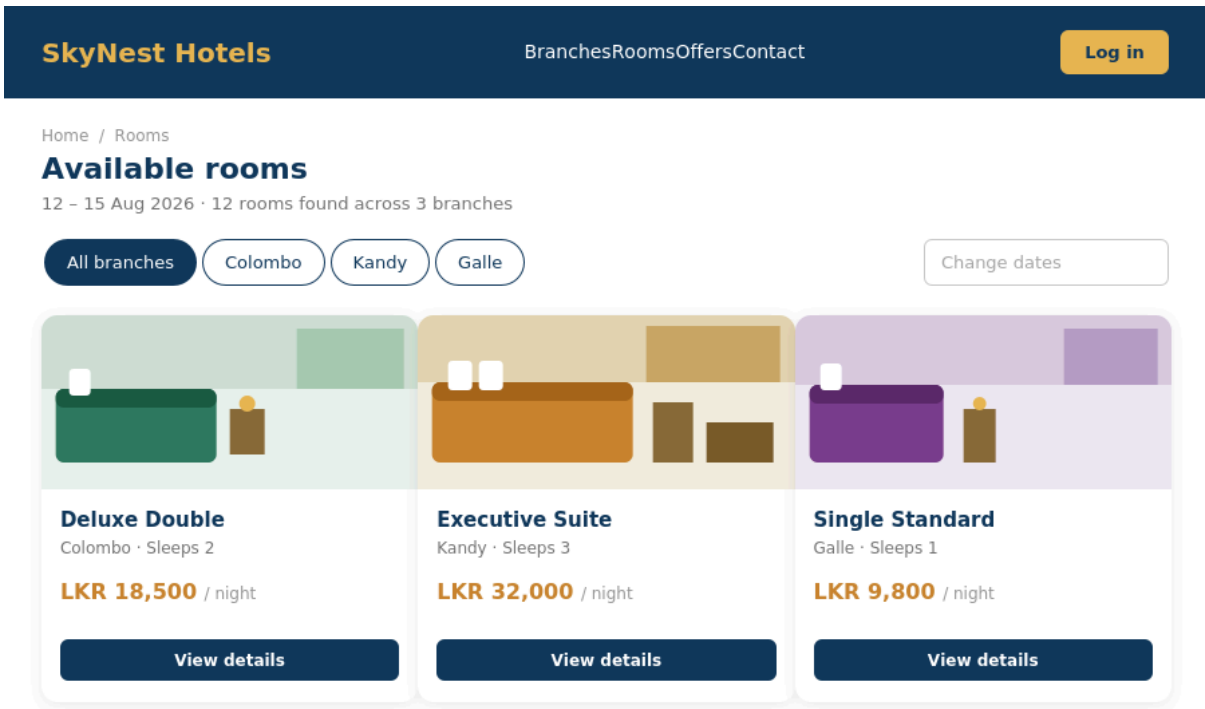
1. Homepage

The landing page introduces SkyNest Hotels and allows any visitor to search for rooms by branch and date without needing to log in. It highlights the three branch locations (Colombo, Kandy, Galle) to help guests choose a location before browsing available rooms.



2. Room Browsing & Search

Guests can browse rooms by branch, filter by dates, and view room cards showing type, capacity, and nightly rate. This screen is accessible without an account, consistent with the requirement that guests may browse freely and are only prompted to log in when they attempt to book.



3. Login & Account Access

The login screen authenticates all user types (guest, receptionist, manager, admin) through a single form. The system determines the user's role from their credentials and opens the corresponding dashboard. Guests who do not yet have an account can create one directly from this screen, or continue booking without registering.

The screenshot shows the login interface for SkyNest Hotels. It features a white login card centered on a dark blue background. The card contains the 'SkyNest Hotels' logo and the text 'Log in to continue'. There are two input fields: 'Email' with the placeholder 'name@example.com' and 'Password' with the placeholder 'Enter your password'. Below these fields is a prominent orange 'Log in' button. A horizontal line with the word 'OR' in the center separates the login section from the registration section. The registration section includes the text 'New here? You can book without an account, or create one to track your stay.' and a 'Create a guest account' button.

4. Booking & Checkout

Once a guest selects a room, this screen collects guest details, check-in/check-out dates, and payment method, and displays a real-time price breakdown including room charge, tax, and any applicable discount. Confirming the booking creates a record with status "Booked" and generates the associated invoice.

SkyNest Hotels

Step 2 of 3 · Booking details

Your selected room



Deluxe Double · Colombo
Sleeps 2 · Sea view · Free Wi-Fi
Check-in: 12 Aug 2026 · Check-out: 15 Aug 2026 · 3 nights

Guest details

Full name

Your full name

Phone

+94 7X XXX XXXX

Email

name@example.com

ID / passport no.

National ID or passport

Payment method

Credit / debit card

Cash at check-in

Bank transfer

Booking summary

Room charge (3 nights)	LKR 55,500
Tax (10%)	LKR 5,550
Discount	– LKR 2,000
Total due	LKR 59,050

Confirm booking

5. Service Catalogue & Request Services

During their stay, guests can browse the service catalogue — room service, spa treatment, laundry, minibar, airport transfer, and gym day pass — and add them to their booking directly from this screen. Each selection is recorded as a service usage entry (service, date, quantity, and price at the time of use) and reflected immediately in a running bill alongside the room charge, which is finalized at checkout.

Add services to your stay

Charges are added to your booking and settled at checkout

 Room service LKR 1,200 / order	- 2 +
 Spa treatment LKR 4,500 / session	- 1 +
 Laundry LKR 800 / load	- 0 +
 Minibar LKR 950 / item	- 3 +
 Airport transfer LKR 3,000 / trip	- 0 +
 Gym day pass LKR 500 / day	- 0 +

Current bill

Room 204 · 12-15 Aug 2026

Room charge (3 nights)	LKR 55,500
Room service ×2	LKR 2,400
Spa treatment ×1	LKR 4,500
Minibar ×3	LKR 2,850

Running total LKR 65,250**Save services to bill**

6. Receptionist Dashboard

Receptionists use this dashboard to manage day-to-day front-desk operations: viewing today's arrivals and departures, checking guests in (which updates room status to Occupied), logging service usage against a booking, processing full or partial payments, and checking guests out (which calculates the final bill and updates room status back to Available)

Receptionist dashboard — Colombo branch

RN

Arrivals today

6

Departures today

4

Outstanding dues

LKR 42,000

Today's bookings

Guest	Room	Status	
A. Perera	204 · Double	Booked	Check in
N. Silva	310 · Suite	Occupied	Log service
R. Fernando	112 · Single	Dues pending	Check out

7. Manager Dashboard

Managers can view the five required reports — room occupancy, guest billing summary, service usage breakdown, monthly revenue, and top-used services/trends — scoped to their own branch only.

Manager dashboard — Kandy branch This month ▼

Occupancy rate
78%

Revenue (month)
LKR 4.2M

Unpaid balances
LKR 96,000

Top service
Spa

Reports

● Room occupancy

● Billing summary

● Service usage

● Monthly revenue

● Top services & trends

8. Admin Dashboard

Admins have the same five reports as managers, but scoped across all branches, giving a consolidated view of the entire hotel chain's performance. Admins also manage staff accounts directly from this dashboard: viewing existing manager and receptionist accounts by branch, and creating new accounts through an inline form (name, role, branch, and email)

Admin dashboard — all branches This month ▼

Occupancy rate
82%

Revenue (all branches)
LKR 11.6M

Unpaid balances
LKR 210,000

Top service
Spa

Reports (all branches)

● Room occupancy

● Billing summary

● Service usage

● Revenue per branch

● Top services & trends

Manage manager & receptionist accounts + Create account

Name	Role	Branch	Email		
S. Wickramasinghe	Manager	Colombo	s.wick@skynest.lk	Edit	Disable
R. Nanayakkara	Receptionist	Colombo	r.nana@skynest.lk	Edit	Disable
K. Jayasuriya	Manager	Kandy	k.jaya@skynest.lk	Edit	Disable
D. Ranasinghe	Receptionist	Galle	d.rana@skynest.lk	Edit	Disable

Full name

Role

Branch

Email

Create

Staff full name

Manager

Colombo

name@skynest.lk

3.2 Hardware Interfaces

The HRGSMS interacts with various hardware components across front-desk operations, guest-facing devices, and back-office infrastructure to support reservations, check-in/check-out, and payment processing.

Logical Characteristics

1. Device Compatibility & Responsiveness

- The system runs on desktop/laptop computers at reception desks and manager offices, and on tablets used by housekeeping or receptionists for mobile check-in.
- Guests access booking and billing screens via desktop browsers, tablets, or smartphones.
- The UI adapts responsively across these device types with touch-friendly controls for tablet-based front-desk use.

2. Peripheral Devices

- Card payment terminals (POS machines) — for processing credit/debit card payments at checkout.
- Barcode/QR scanners — for scanning ID documents or key cards during check-in.
- Receipt/invoice printers — for printing final bills at checkout.
- Key card encoders — for issuing/re-issuing electronic room key cards during check-in.

3. Control & Connective Features

- Automatic reconnection — if internet connectivity drops at a branch, the system should cache pending check-ins/payments locally and sync once reconnected.
- Power optimization — reception tablets minimize background activity when idle to conserve battery.
- Offline resilience — critical operations (check-in, check-out) should degrade gracefully rather than fail completely during short outages.

4. Communication Protocols

- **HTTPS** — for secure web-based transactions between client devices and the server.
- **Bluetooth/USB** — for connecting card readers, receipt printers, and key card encoders to reception terminals.
- **Wi-Fi/Ethernet** — for networking front-desk computers and tablets within each branch.

Hardware Specifications

- **Processor:** Backend server requires an x86-64 CPU; client devices need at least a dual-core 1.5GHz processor for smooth operation.

- **Storage:** Central database runs on SSD storage for fast query response across bookings, service usage, and billing records; local caching on reception tablets supports brief offline operation.
- **Input/Output:** Supports touchscreens, keyboards, card payment terminals, receipt printers, and barcode/ID scanners.
- **OS Compatibility:** Backend — Linux or Windows server; Frontend — modern web browsers on Windows, macOS, and mobile OS (Android/iOS) for tablet-based front-desk use.
- **Network:** Each branch requires a stable internet connection (minimum recommended bandwidth for real-time booking sync across branches, to prevent double-booking conflicts).

3.3 Software Interfaces

The HRGSMS integrates the following software components. Each interface is described in terms of the data exchanged, the protocol used, and the direction of communication.

Operating Systems: The system's backend API will be deployed on cloud infrastructure (Render.com), while the frontend will be served via a global CDN (Vercel). The application will be accessed via modern web browsers on standard desktop and mobile operating systems, supporting responsive views for user access.

Database Management System (DBMS): The system will use PostgreSQL (version 15+, hosted on Neon.tech) as the relational database. Communication between the backend and the database will occur via the native PostgreSQL wire protocol over SSL/TCP. This bidirectional interface handles SQL statements, stored procedure calls, result sets, and transaction control commands.

Backend Application: A RESTful API will be developed using Python and FastAPI (version 0.100+). This ASGI HTTP framework will handle HTTP request routing, OpenAPI schema generation, and JSON serialization/deserialization. The backend interfaces with the database using the psycopg2 (version 2.9+) Python DB-API 2.0 library, which translates Python objects into SQL parameter bindings, manages cursor result rows, and handles connection pooling.

Frontend Interface: The client-side application, hosted on Vercel, will communicate with the backend API via secure HTTPS REST calls. The data exchanged will consist of JSON request bodies (such as form submissions) and JSON response payloads containing data tables and status messages.

Authentication and Authorization: The system will utilize JSON Web Tokens (JWT) via the python-jose (version 3.3+) library for secure, in-process token validation. Access tokens will be issued upon login and decoded by FastAPI middleware on every protected API request to extract role and user_id claims for authorization verification.

Continuous Integration (CI/CD): GitHub Actions will be utilized to automate testing and deployment workflows. Git push events will trigger webhooks that prompt a CI runner to execute the pytest suite and perform schema migration dry-runs against a dedicated Neon.tech development database branch.

3.4 Communication Interfaces

The system will rely on secure and efficient communication protocols to support data exchange between the frontend client, backend services, the database, and deployment pipelines:

Web Communication

- All client-server communication and static asset delivery (via Vercel CDN) will occur over HTTPS (Port 443) using TLS 1.2+ encryption, with HSTS enabled.
- RESTful APIs will be used to handle requests and responses between the browser and the Render.com FastAPI service.
- API access is secured by a strict Cross-Origin Resource Sharing (CORS) policy via CORSMiddleware, restricting origins strictly to the production Vercel frontend URL and local development environments.

Database Communication

- The Render.com backend will communicate with the Neon.tech PostgreSQL database using the PostgreSQL wire protocol over secure TCP/IP connections (Port 5432).
- SSL encryption is strictly required and enforced by Neon.tech to protect all database data in transit.
- Connection strings (e.g., DATABASE_URL) are stored securely as environment variables on the backend hosting platform and are never committed to source code.

CI/CD and Deployment Communication

- Automated deployments will be triggered via secure HTTPS webhooks (Port 443) sent from GitHub Actions to Render.com upon pushes to the main branch.
- The CI pipeline will securely connect to a dedicated Neon.tech development database branch over SSL to perform schema migration dry-runs.
- All deployment hooks and pipeline credentials will be securely stored and validated as GitHub Actions secrets.

CORS Policy

- Cross-Origin Resource Sharing (CORS) is configured on the FastAPI backend via `fastapi.middleware.cors.CORSMiddleware`.
- Allowed origins: the production Vercel frontend URL and `localhost:5500` (local development). All other origins are rejected.
- Allowed methods: GET, POST, PUT, PATCH, DELETE, OPTIONS.

- Allowed headers: Authorization, Content-Type.

Authentication Flow

- Login: POST /auth/login with {email, password} → FastAPI validates credentials against user_account (bcrypt hash comparison) → issues a signed JWT (HS256, 8-hour expiry) containing {user_id, role, branch_id}.
- Protected endpoints: the JWT is passed as Authorization: Bearer <token> header. FastAPI middleware decodes and validates the token on every request; role and branch_id claims are used for access control and row-level filtering.
- Token refresh: not implemented in the initial release; users must re-authenticate after expiry.

Data Format

- All API payloads are JSON-encoded (Content-Type: application/json).
- Dates are transmitted as ISO 8601 strings (YYYY-MM-DD for dates; YYYY-MM-DDTHH:MM:SSZ for timestamps).
- Monetary values are transmitted as JSON numbers with exactly two decimal places.
- Enum values (booking_status, room_status, payment_method, role) are transmitted as lowercase strings matching the PostgreSQL CHECK constraint values.

Error Communication

- Database constraint violations (e.g., double-booking trigger, outstanding balance block) are caught by psycopg2 as psycopg2.errors.RaiseException and translated into 409 Conflict or 400 Bad Request HTTP responses with a descriptive message field.
- Unhandled exceptions are logged server-side (to Render.com log stream) and return a generic 500 response. PII (NIC/passport numbers) is scrubbed from log output by a custom logging filter.

4.0 System Features

This section organises the functional requirements by system feature, the major services the product must provide. The most critical transactions within each feature are additionally documented as a Detailed Use Case Specification, following the structured table format used in the reference example.

4.1 Room and Branch management

4.1.1 Description and Priority

High Priority – Enables administrators to manage hotel branches, room types, and individual rooms. Ensures room data and pricing are maintained accurately without disrupting historical records.

4.1.2 Stimulus/Response Sequences

- Administrator registers a new branch → System creates branch record with location, contact, and manager details.
- Administrator adds a room → System links room to branch and room type; sets initial status to Available.
- Administrator updates room type daily rate → System updates rate; existing confirmed bookings are not retroactively affected.

4.1.3 Functional Requirements

REQ-1.1: The system shall store details for each hotel branch (e.g., name, location, and manager).

REQ-1.2: The system shall maintain various room types along with their capacities, pricing, and amenities.

REQ-1.3: The system shall track individual rooms, linking them to a specific branch and monitoring their current status.

REQ-1.4: The system shall allow authorized staff to manage rooms and room types without altering past booking records.

REQ-1.5: The system shall provide a clear view of current room availability across all branches.

4.2 Booking Management

4.2.1 Description and Priority

High Priority – Allows staff to handle room reservations securely, managing the lifecycle of a booking from creation to cancellation while strictly preventing double-bookings.

4.2.2 Stimulus/Response Sequences

- Staff enters booking dates and details → System verifies availability and confirms the reservation.
- Staff cancels a standard booking → System updates the status and frees up the room.
- Staff cancels a checked-in booking → System applies the required no-show policy and corresponding charges.

4.2.3 Functional Requirements

REQ-2.1: The system shall allow staff to create a booking capturing guest, room, check-in date, check-out date, payment method, and an initial status of Booked.

REQ-2.2: The system shall reject any new or modified booking whose date range overlaps an existing active (Booked or Checked-In) booking for the same room.

REQ-2.3: The system shall support booking statuses Booked, Checked-In, Checked-Out, and Cancelled, and shall constrain valid transitions between them (e.g., Checked-Out cannot revert to Booked).

REQ-2.4: The system shall allow a Booked reservation to be cancelled prior to check-in, releasing the room for that date range.

REQ-2.5: The system shall associate every booking with a guest profile, supporting repeat guests across multiple bookings.

4.3 Guest Management

4.3.1 Description and Priority

Guest records are required before any booking can be created. The system must support both individual and corporate (organisational) guests. Priority: High.

4.3.2 Stimulus/Response Sequences

- Front desk registers a new guest → System validates NIC/Passport uniqueness and stores guest record.
- Front desk searches for an existing guest by name or ID → System returns matching guest records.
- Guest completes their 5th booking → System flags the guest for the 5% loyalty discount on the next booking.

4.3.3 Functional Requirements

REQ-3.1: The system shall store guest records with: full name, NIC or passport number (unique), date of birth, nationality, contact phone, email address, and guest type (Individual / Corporate).

REQ-3.2: For Corporate guests, the system shall additionally store: company name, company registration number, and billing contact name.

REQ-3.3: The system shall enforce uniqueness of NIC/Passport number across all guest records.

REQ-3.4: The system shall track the total number of completed bookings per guest to support loyalty discount eligibility checks.

REQ-3.5: The system shall allow guest records to be searched by name, NIC/Passport, or contact number.

4.4 Check-In and Check-Out

4.4.1 Description and Priority

Room status must accurately reflect physical occupancy. Status transitions must be atomic and trigger-driven. Priority: High.

4.4.2 Stimulus/Response Sequences

- Front desk performs check-in on a Booked booking → System updates booking status to Checked-In; trigger updates room status to Occupied.
- Front desk initiates check-out → System invokes billing procedure → If outstanding balance > 0, checkout is blocked until payment is posted → On full payment, status transitions to Checked-Out; trigger updates room status to Available.

4.4.3 Functional Requirements

REQ-4.1: The system shall implement check-in as a stored procedure that: (a) validates the booking is in Booked status, (b) sets booking status to Checked-In, (c) sets the room status to Occupied — all within a single transaction.

REQ-4.2: An AFTER UPDATE trigger on the Booking table shall automatically update the corresponding Room.status to Occupied when booking status changes to Checked-In, and to Available when it changes to Checked-Out or Cancelled.

REQ-4.3: The system shall implement checkout as a stored procedure that: (a) calculates the total bill (room charges + service charges + tax), (b) checks the outstanding balance, (c) blocks status transition to Checked-Out if outstanding_balance > 0, (d) on zero balance, sets booking status to Checked-Out.

REQ-4.4: Late checkout (after 12:00 noon, if tracked) shall incur a 50% additional daily rate charge, appended to the bill as a service line item.

4.5 Chargeable Service Tracking

4.5.1 Description and Priority

Service charges are a significant revenue stream. Historical price integrity (price at time of use) must be preserved. Priority: High.

4.5.2 Stimulus/Response Sequences

- Front desk logs a service request for an active booking → System records service, quantity, unit price (from catalogue at time of entry), and timestamp.
- Service catalogue price is updated → Previously logged service usage records are NOT affected.

4.5.3 Functional Requirements

REQ-5.1: The system shall maintain a Service Catalogue with: service_id, service_name, category (Room Service / Spa / Laundry / Minibar / Transport / Other), unit price (LKR), and description.

REQ-5.2: The system shall record service usage entries linked to a booking: usage_id, booking_id, service_id, usage_date, quantity, and unit_price_at_usage (snapshot of price at time of logging).

REQ-5.3: A BEFORE INSERT trigger on ServiceUsage shall validate that the linked booking is in Checked-In status before allowing the insert. Usage entries on Booked or Checked-Out bookings shall be rejected.

REQ-5.4: The system shall maintain 6 service entries in the catalogue at minimum: (1) Room Service, (2) Spa Treatment, (3) Laundry, (4) Minibar, (5) Airport Transfer, (6) Extra Bed.

REQ-5.5: The system shall provide a query/view returning service usage breakdown per booking, aggregated by service category.

4.6 Billing and Payments

4.6.1 Description and Priority

Accurate billing is the primary deliverable of the checkout process. The system must handle partial payments and flag outstanding dues. Priority: High.

4.6.2 Stimulus/Response Sequences

- Billing procedure invoked at checkout → Calculates room charges (rate × nights) + service charges (sum of usage) + 15% VAT + 1% Tourism Levy – applicable discount → Produces itemised bill.
- Front desk records a partial payment → System updates amount_paid; computes outstanding_balance; sets balance_flag.
- Checkout attempted with outstanding balance > 0 → Trigger/procedure raises exception and blocks the Checked-Out status transition.

4.6.3 Functional Requirements

REQ-6.1: The system shall maintain a Bill table linked to each booking, storing: bill_id, booking_id, room_charges, service_charges, discount_amount, tax_amount (15% VAT + 1% Tourism Levy), total_amount, amount_paid, outstanding_balance, balance_flag (Boolean).

REQ-6.2: Room charges shall be calculated as: (room daily rate × number of nights) – loyalty discount (if applicable).

REQ-6.3: Tax shall be calculated as: (room_charges + service_charges – discount) × 0.16 (combined 15% VAT + 1% levy).

REQ-6.4: A stored procedure (calculate_bill) shall compute and insert/update the Bill record for a given booking_id. It shall be callable at any time to produce a live bill estimate.

REQ-6.5: A stored procedure (record_payment) shall accept a booking_id and payment_amount, update amount_paid, recompute outstanding_balance, and set balance_flag = TRUE if outstanding_balance > 0.

REQ-6.6: A BEFORE UPDATE trigger on the Booking table shall prevent transition to Checked-Out status if the corresponding Bill.outstanding_balance > 0.

REQ-6.7: The system shall maintain a Payment table recording each payment transaction: payment_id, booking_id, payment_date, amount, payment_method.

4.7 Management Reports

4.7.1 Description and Priority

Reports are the primary decision-support output for hotel management. Priority: Medium-High.

4.7.2 Stimulus/Response Sequences

- Manager selects report type and date range → System executes the corresponding stored procedure or view → Results returned in tabular form.

4.7.3 Functional Requirements

REQ-7.1: Room Occupancy Report: Given a date or date range and an optional branch filter, return each room with its status, booking reference (if occupied), guest name, and check-in/check-out dates.

REQ-7.2: Guest Billing Summary: For a selected guest or all guests, return booking history, total bill, amount paid, outstanding balance, and balance flag. Flag records with balance_flag = TRUE prominently.

REQ-7.3: Service Usage Breakdown: For a selected booking or date range, return aggregated service usage by service category, showing total units consumed and total charge per category.

REQ-7.4: Monthly Revenue Report: For a selected month and optional branch filter, return total room revenue, total service revenue, total tax collected, total discounts granted, and net revenue. Grouped by branch.

REQ-7.5: Top Services Report: Return the top-N services ranked by total revenue generated in a selected period. Include usage count and average revenue per usage.

5.0 Non-Functional Requirements

1. Performance

- The system shall display room availability within a few seconds.

- Booking, check-in, and check-out transactions shall complete within a few seconds under normal load.
- Reports shall be generated within 5 seconds under normal operating conditions.

2. Reliability

- The database shall maintain ACID properties for all transactions.
- The system shall prevent data corruption during unexpected failures.
- No booking information shall be lost due to system crashes.

3. Availability

- The system shall be available 24/7 except during scheduled maintenance.

4. Security

- The system shall require user authentication for hotel staff.
- Guests shall be able to create, view, and manage their own bookings through the user interface.
- Hotel staff shall be authenticated using usernames and passwords before accessing administrative functions.
- Sensitive customer information (e.g., personal details and payment information) shall be protected from unauthorized access.
- The system shall implement role-based access control with different permissions for:
 - Guest – Make, view, modify, or cancel their own bookings.
 - Receptionist – Manage bookings, check-ins/check-outs, and payments.
 - Manager – View reports and oversee hotel operations.
 - Administrator – Manage users, branches, rooms, and system settings.

5. Data Integrity

- Duplicate bookings for the same room during overlapping dates shall not be allowed.
- Invalid or inconsistent records shall be prevented using constraints and triggers.

6. Usability

- The user interface shall be simple and easy for hotel staff to operate.
- Error messages shall clearly explain the problem (e.g., "Room already booked for selected dates").

7. Consistency

- Room status shall always reflect the actual booking status.
- Billing information shall remain consistent after every payment transaction.

8. Performance Optimization

- Frequently searched columns (RoomID, GuestID, BookingID, BranchID, CheckInDate) shall be indexed.
- Reports should use optimized SQL queries to minimize execution time.

6.0 Other Requirements

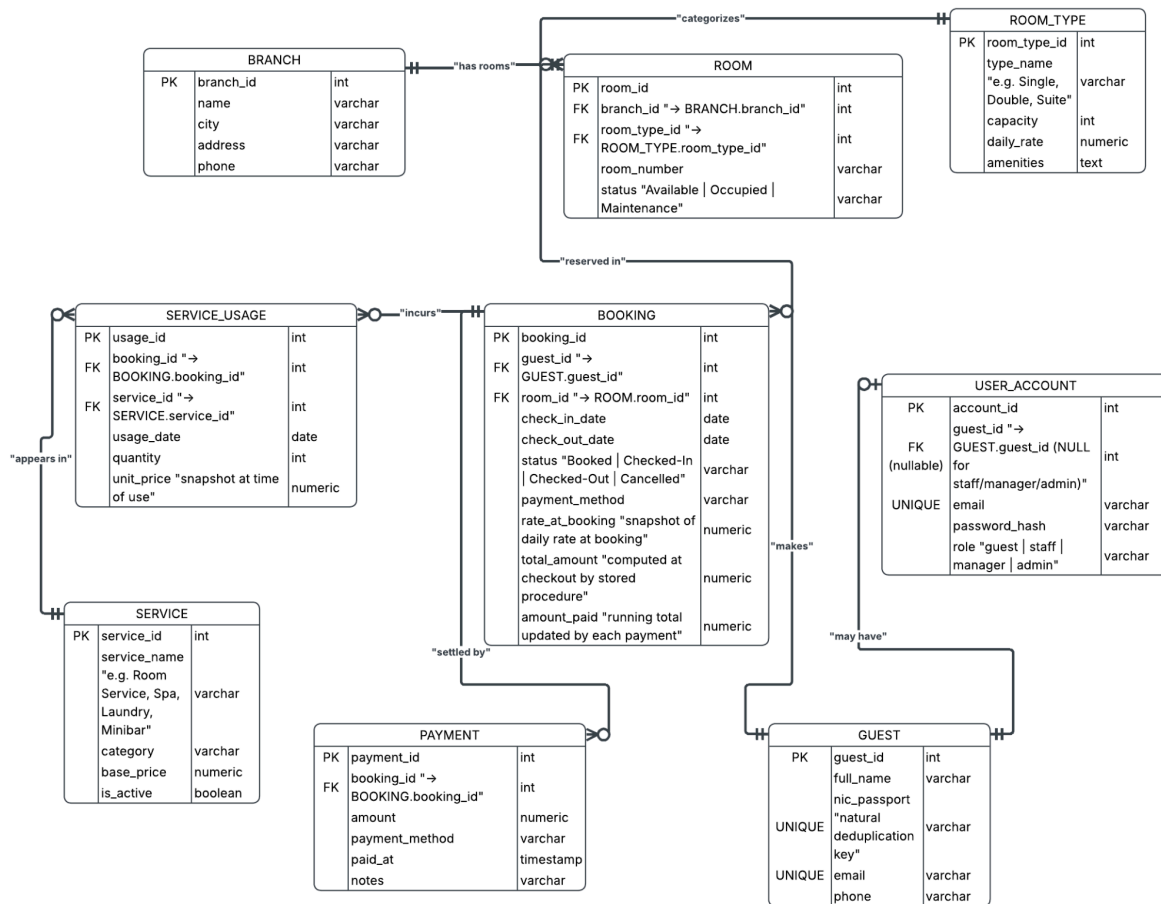
Appendix A : Glossary

Term	Definition
API	Application Programming Interface, a set of rules and protocols that allows different software applications to communicate with each other
DBMS	Database Management System is software that is used to create, manage, and manipulate databases
IEEE	Institute of Electrical and Electronics Engineers, an organization that standardizes and maintains modern standards of electrical, computer science, and electronic engineering fields
MySQL	A relational database management system provided by Oracle.
SRS	Software Requirements Specification, a documented overview of the software's functionality and operational principles, delineating the various stakeholders who play distinct roles in its development and utilization

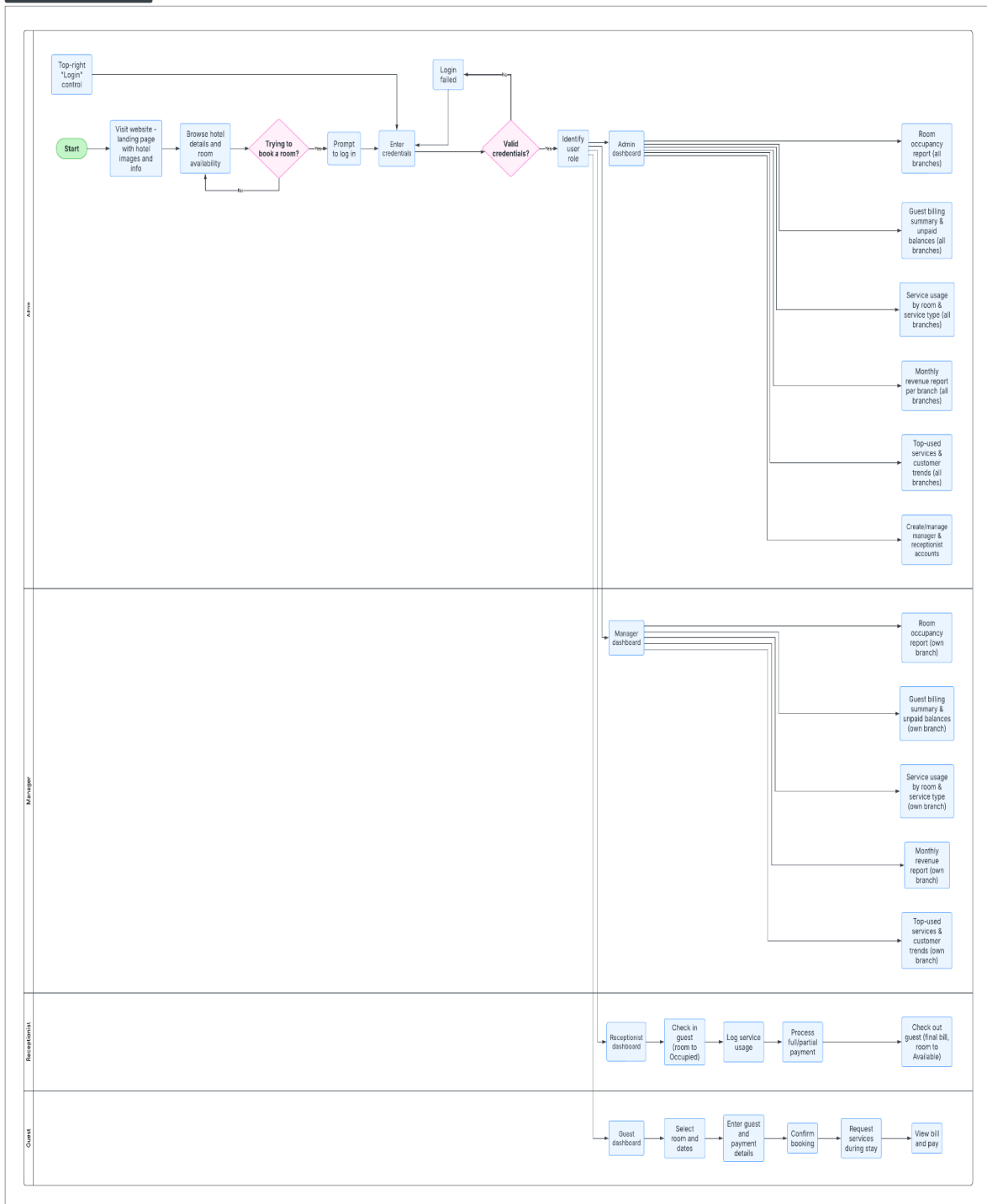
RLS	Row Level Security, a database security feature that restricts which rows a user can access or modify based on their role, enforced at the database level rather than in application code
MVCC	Multi-Version Concurrency Control, a database concurrency control method that allows multiple transactions to read and write data simultaneously without locking, by maintaining multiple versions of a row
Database Schema	The formal structure that defines how data is organized in a database, including tables, columns, data types, primary/foreign keys, and the relationships between them
PoC	Proof of Concept, a small-scale implementation built to demonstrate that a proposed design or feature
BCNF	Boyce-Codd Normal Form, a stricter version of Third Normal Form (3NF) in database normalization, ensuring that every determinant in a table is a candidate key — used to eliminate redundancy and update anomalies when designing tables like Booking, Invoice, and Service_Usage.
HRGSMS	Hotel Reservation and Guest Services Management System, the system being developed in this project to manage room bookings, guest check-in/check-out, service tracking, and billing across SkyNest Hotels' branches.
CDN	A Content Delivery Network is a group of connected servers placed in different parts of the world. It makes websites load faster by storing copies of website files close to where users live

Appendix B : Analysis Models

Entity-Relationship diagram



Workflow diagram



Appendix C : To Be Determined

Late checkout time tracking — Whether actual checkout time is captured via the system clock automatically or requires manual staff input. Affects trigger design for the 50% half-day surcharge. *(TBD before trigger implementation phase.)*

Report export format — Whether CSV export from the Reports screen is in scope for the PoC demo or deferred to a future phase. *(To be confirmed by project supervisor before UI development.)*

Room rate seasonality — Whether peak/off-peak pricing tiers will be supported in the initial release. Currently assumed fixed rates. *(Requires confirmation from SkyNest Hotels management.)*

Multi-room bookings — Whether a single booking can cover multiple rooms (e.g., for corporate guests). Currently out of scope; one booking maps to exactly one room. *(To be confirmed with supervisor.)*

Loyalty discount tracking mechanism — Whether completed booking count is computed live via a query on each booking creation, or maintained as a denormalised counter on the guest record. Affects stored procedure design. *(To be resolved before billing procedure implementation.)*

Authentication mechanism — JWT stateless tokens vs. server-side session cookies for the FastAPI backend. Affects API contract and deployment configuration. *(To be decided before backend development begins.)*

Partial payment refund handling — Whether refunds for cancelled bookings (post check-in) are tracked within the system or handled externally. Currently out of scope. *(To be confirmed before cancellation procedure is finalised.)*

Appendix D : Team Members

Index Number	Name
240159J	Fernando H. T. S. D.
240191A	Ginige V. T.
240621U	Sheereen Humaa M. R.
240006N	Abemal A. M. S. P.
240107B	De Zoyza P. C. A.